

1.INTRODUCTION

Agriculture plays a vital role in the economic development of many countries, especially in regions where farming is the primary source of livelihood. One of the major challenges faced by farmers is effective water management. Traditional irrigation methods often lead to overwatering or underwatering, which negatively impacts crop yield and wastes valuable water resources. With increasing water scarcity and unpredictable rainfall patterns, there is a growing need for intelligent irrigation systems that can operate automatically based on environmental conditions.

The Automatic Irrigation System Using Rain Sensor and Soil Moisture Sensor is developed to address these challenges by implementing smart sensing and automated control mechanisms. The system monitors soil moisture content using a soil moisture sensor that measures the volumetric water content in the soil. When the soil becomes dry and moisture falls below a certain threshold level, the microcontroller activates the water pump to irrigate the field. Once the desired moisture level is restored, the pump is automatically switched off.

In addition to soil monitoring, the system incorporates a rain sensor that detects rainfall. If rain is detected, the irrigation process is temporarily suspended to prevent water wastage. This ensures efficient utilization of natural resources and reduces dependency on manual supervision.

The integration of sensors with a microcontroller-based control system makes the irrigation process smart and reliable. The automation reduces labor costs and ensures timely watering, which is crucial for healthy crop growth. The system can be implemented in agricultural fields, gardens, and greenhouses. It is affordable, easy to install, and requires minimal maintenance.

By adopting such automated irrigation techniques, farmers can improve crop productivity while conserving water and energy. This project represents a step toward precision agriculture and sustainable farming practices, contributing to better resource management and increased agricultural efficiency.

1.1 Organization Profile

NOBEL SOFTWARE SOLUTIONS is a professional website designing, Customized software development, Business process Outsourcing – IT/ITES & Internet marketing company providing full featured web service including B2A Acquisition & B2C e-commerce solutions and acting as an offshore development center for overseas development firms. NOBEL SOFTWARE SOLUTIONS is an innovative company, based in India that provides a series of Web-based and software and software applications that have helped their customer to create successful business ventures through online initiatives. NOBEL SOFTWARE SOLUTIONS provide all the services that a company needs to get online from web designing to web hosting and manage leading-edge web sites and e-business software solutions success in the domestic market. Ever since its inception, NOBEL software solutions has accrued continuous growth in all its business functions and this has been possible only due to its commitment, quality training methodologies, the services it offers, knowledge sharing with industry leaders and professional approach.

Services

- Professional Web Design
- SEO concerts, Internet Marketing
- Pay per click Campaign
- Link Building, Ecommerce Solution
- Web Application Development
- Multimedia Presentations
- Customized Software development
- Business Process Outsourcing-IT/IT.

1.2.SYSTEM SPECIFICATION

1.2.1.Hardware Requirements

For IoT Device Setup:

- Microcontroller : ESP8266 (NodeMCU)
- Sensor : Water level sensor, Moisture Sensor
- Alert System : Buzzer / LED Indicator
- Power Supply : 5V DC Adapter / USB Power
- Connectivity : Wi-Fi Network
- Additional Components : Jumper Wires, Breadboard / PCB

For Monitoring System:

- Processor : Intel Dual Core or above
- Processor Speed : 2.0 GHz or higher
- RAM : 4 GB RAM (Minimum 2 GB)
- Hard Disk : 250 GB or above
- Monitor : 15-inch Color Monitor or higher
- Input Devices : Standard Keyboard and Scroll Mouse
- Smartphone : Android / iOS device with internet access (for Blynk App monitoring)

1.2.2.Software Specification

- Front end : Blynk App
- Back end : NodeMCU ESP8266 Programmable board
- Programming Language : Embedded C / Arduino IDE
- IoT Platform : Blynk IoT Platform
- Microcontroller IDE : Arduino IDE
- Operating System : Windows 10 or above
- Mobile Application : Blynk App (Android / iOS)
- Browser (Optional) : Google Chrome / Edge for Blynk Web Dashboard

1.3. SOFTWARE DESCRIPTION

Introduction to front end

Python:

Python is a high-level, interpreted and general-purpose dynamic programming language that focuses on code readability. The syntax in Python helps the programmers to do coding in fewer steps as compared to Java or C. Python is widely used in bigger organizations because of its multiple programming paradigms. They usually involve imperative and object-oriented functional programming. It has a comprehensive and large standard library that has automatic memory management and dynamic features.

The software development companies prefer Python language because of its versatile features and fewer programming codes. Nearly 14% of the programmers use it on the operating systems like UNIX, Linux, Windows and Mac OS. Interactive

Advantages or Benefits of Python

The Python language has diversified application in the software development companies such as in gaming, web frameworks and applications, language development, prototyping, graphic design applications, etc. This provides the language a higher plethora over other programming languages used in the industry. Some of its advantages are-

Extensive Support Libraries

It provides large standard libraries that include the areas like string operations, Internet, web service tools, operating system interfaces and protocols. Most of the highly used programming tasks are already scripted into it that limits the length of the codes to be written in Python.

Presence of Third Party Modules:

The Python Package Index (PyPI) contains numerous third-party modules that make Python capable of interacting with most of the other languages and platforms.

Integration Feature

Python integrates the Enterprise Application Integration that makes it easy to develop Web services by invoking COM or COBRA components. It has powerful control capabilities as it calls directly through C, C++ or Java via Jython. Python also processes XML and other markup languages as it can run on all modern operating systems through same byte code.

Improved Programmer's Productivity

The language has extensive support libraries and clean object-oriented designs that increase two to ten fold of programmer's productivity while using the languages like Java, VB, Perl, C, C++ and C#.

Productivity

With its strong process integration features, unit testing framework and enhanced control capabilities contribute towards the increased speed for most applications and productivity of applications. It is a great option for building scalable multi-protocol network applications.

Open Source and Community Development:

Python language is developed under an OSI-approved open source license, which makes it free to use and distribute, including for commercial purposes.

Further, its development is driven by the community which collaborates for its code through hosting conferences and mailing lists, and provides for its numerous modules.

Learning Ease and Support Available:

Python offers excellent readability and uncluttered simple-to-learn syntax which helps beginners to utilize this programming language. The code style guidelines, PEP 8, provide a set of rules to facilitate the formatting of code. Additionally, the wide base of users and active developers has resulted in a rich internet resource bank to encourage development and the continued adoption of the language.

User-friendly Data Structures:

Python has built-in list and dictionary data structures which can be used to construct fast runtime data structures. Further, Python also provides the option of dynamic high-level data typing which reduces the length of support code that is needed.

Simple and easy to Learn :

- The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily.
- The elegant syntaxes of the python language makes the people to learn python in easiest manner.
- Without having any other programming languages knowledge, we can learn python directly.
- The simple and powerful syntax of python language makes the programmers to express their business logic in less lines of code.

Platform Independent :

- Like Java programs, Python programs are also platform independent.
- Once we write a Python program, it can run on any platform without rewriting once again.
- Python uses PVM to convert python code to machine understandable code.

High Level Language :

- Python is a high level language.
- While developing python applications, developers no need to bother about memory management.

Dynamically Typed Language :

- Python is a dynamically typed language. I mean, in python there is no need to declare type of a variable.

- Whenever we assign a value to the variable, based on the value the type will be allocated automatically.

Python supports POP & OOP :

- Python language supports both Procedural Oriented Programming and Object Oriented Programming features.
- We can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in python programming language.

Python is interpreted Language :

- Like PHP, Python is also an interpreted language.
- Python applications do not require explicit compilation so that a compiler is not required for python software.
- Python interpreter is responsible for execution of python applications.
- Whenever we run a python application, the python interpreter checks for the syntax errors. If there are no syntax errors found in our code then the interpreter converts the code into intermediate code in the form of low level format and executes it.
- The intermediate code of the python applications is known as byte code.
- The extension of the byte code file is **.pyc** (Python Compile Code).

Python is Embeddable :

- We can embed the python code into other languages such as C,C++, Java and Etc..
- In order to provide the scripting capabilities to other languages we can use the python code in it.

Develop GUI & Web Application :

- We can develop GUI based applications using Python Languages.
- We can also develop Web applications using Python Language.

Applications of Python:

- GUI based desktop applications
- Image processing and graphic design applications
- Scientific and computational applications
- Games
- Web frameworks and web applications
- Enterprise and business applications
- Operating systems
- Language development
- Prototyping

2.SYSTEM STUDY

2.1. EXISTING SYSTEM

In the existing irrigation system, farmers manually monitor soil conditions and control water supply. Irrigation is usually performed at fixed time intervals without considering real-time soil moisture or weather conditions. In some cases, basic timer-based systems are used, which activate the pump for a set duration regardless of the soil's actual requirement. These traditional methods often lead to over-irrigation or under-irrigation. Water wastage is common, especially during rainfall when irrigation continues unnecessarily. Manual monitoring also increases labor costs and requires constant supervision. The lack of automation and environmental sensing makes existing systems inefficient and less reliable for modern agricultural needs.

2.1.1. Drawbacks

- High water wastage due to lack of monitoring.
- Overwatering or underwatering of crops.
- Requires continuous manual supervision.
- Increased labor costs.
- No rainfall detection mechanism.
- Reduced crop productivity due to inconsistent watering.

2.2. PROPOSED SYSTEM

The proposed system introduces an automated irrigation mechanism using a soil moisture sensor and a rain sensor. The soil moisture sensor continuously monitors the moisture level in the soil. If the moisture drops below a predefined threshold, the microcontroller activates a relay to turn on the water pump. When sufficient moisture is reached, the pump is automatically turned off. The rain sensor detects rainfall and stops irrigation during rainy conditions. This system

ensures optimal water usage and eliminates manual intervention. It improves efficiency, conserves water, reduces labor, and supports sustainable agricultural practices through smart and automated irrigation control.

2.2.1. Features

- Automatic soil moisture monitoring.
- Rain detection for smart irrigation control.
- Microcontroller-based automation.
- Water-saving mechanism.
- Low power consumption.
- Easy installation and maintenance.

3.SYSTEM DESIGN AND DEVELOPMENT

SYSTEM DESIGN

3.1 File Design

The file design of the system organizes all project resources into a well-structured directory hierarchy to ensure easy maintenance and scalability. The main application file (app.py) controls routing, request handling, and interaction with the Stable Diffusion model. Template files stored inside the templates folder define the user interface layouts, while static resources such as CSS stylesheets, uploaded images, and generated results are stored inside the static directory. Model-related files such as the Stable Diffusion .safetensors file and CLIP model files are placed inside a separate models folder to clearly separate application logic from large model data.

The system also includes a local SQLite database file (database.db) that stores generation metadata such as prompts, settings, and file paths of output images. Uploaded images are saved inside static/uploads, and generated images are saved inside static/results. This file organization improves access speed, simplifies debugging, and ensures that different components of the system do not interfere with each other.

3.2 Input Design

The input design focuses on collecting user data in a structured and user-friendly manner through HTML forms. The main inputs include text prompt, style selection, lighting selection, camera view, background description, negative prompt, generation steps, and guidance scale. Dropdown menus and sliders are used for parameter selection to reduce user errors. All inputs are validated on the server side before processing to ensure correct data types and acceptable ranges.

For Image-to-Image generation, an additional file input field allows users to upload a base image. The system checks the file format, size, and validity before saving it to the server.

This validation process prevents invalid or malicious files from being processed and ensures stable system operation.

3.3 Output Design

The output design presents the generated images clearly on the web interface after processing. Each generated image is displayed along with options to download the image. The system also shows generation details such as prompt, selected parameters, and generation time, which helps users understand how the output was produced.

In addition to real-time display, the system stores all generated image paths and metadata in the SQLite database. A history page allows users to view previous generations, revisit settings, and delete unwanted records. This improves usability and enables users to track their work efficiently.

3.4 Code Design

The code design follows a modular and layered approach. Separate functions are used for database handling, input validation, model loading, and image generation. This improves readability and makes the system easier to maintain and extend. The Stable Diffusion model is loaded once and reused for multiple requests to reduce overhead.

Error handling mechanisms are implemented using try-except blocks to capture runtime issues such as model loading failures or invalid inputs. The application uses Flask routing to manage different pages such as generation, history, and result view. This structured code design ensures reliability, reusability, and scalability of the system.

3.5 SYSTEM DEVELOPMENT

3.5.1. Description of Modules

1. Soil Moisture Sensor Module

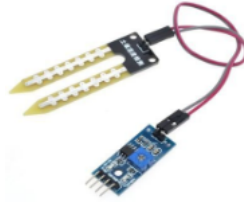


Fig 1: Soil Moisture Sensor (YL-69)

The Soil Moisture Sensor Module is responsible for measuring the water content present in the soil. It works on the principle of electrical conductivity, where the resistance between the sensor probes changes based on the moisture level. When the soil is dry, resistance is high; when the soil is wet, resistance decreases. The sensor continuously monitors the moisture condition and generates either analog or digital output signals. These signals are transmitted to the microcontroller for processing. By setting a threshold value, the system determines whether irrigation is required. This module ensures that crops receive water only when necessary, preventing over-irrigation and maintaining optimal soil conditions for healthy plant growth.

2. Rain Sensor Module



Fig 2: Rain Module (YL-83)

The Rain Sensor Module detects the presence of rainfall and prevents unnecessary irrigation during rainy conditions. It typically consists of a rain-sensitive plate that detects water droplets on its surface. When rainwater falls on the sensor, it changes the conductivity across the sensing plate, generating a signal. This signal is sent to the microcontroller, which immediately

stops the irrigation process if the pump is running. The rain sensor helps conserve water by utilizing natural rainfall effectively. It also protects crops from overwatering, which can damage roots and reduce productivity. This module enhances the overall efficiency and intelligence of the irrigation system.

3. Microcontroller Module



Fig 3: NodeMCU ESP8266

The Microcontroller Module acts as the brain of the entire system. It receives input signals from both the soil moisture sensor and the rain sensor. Based on programmed logic and predefined threshold values, it decides whether the water pump should be turned ON or OFF. The microcontroller processes sensor data in real time and sends control signals to the relay module. It ensures accurate decision-making and smooth operation of the system. Additionally, it manages timing functions and power control, making the irrigation process fully automated and reliable.

4. Relay Module

The Relay Module acts as an electrical switch that controls the operation of the water pump. Since the microcontroller operates at low voltage and current levels, it cannot directly drive the high-power pump. The relay acts as an intermediary component, allowing the microcontroller to safely control the pump. When the microcontroller sends a signal, the relay

either completes or breaks the circuit connected to the pump. This ensures safe switching and protects the system from electrical damage.

5. Water Pump Module

The Water Pump Module is responsible for supplying water to the agricultural field. When activated by the relay, the pump draws water from a reservoir or water source and delivers it to the crops through irrigation pipes. The pump operates only when the soil moisture level falls below the required threshold and no rain is detected. This controlled operation ensures efficient water distribution and reduces unnecessary power consumption. The pump plays a vital role in maintaining consistent soil moisture levels for crop growth.

6. Power Supply Module

The Power Supply Module provides the required electrical energy to all system components, including sensors, microcontroller, relay, and pump. It converts the available power source (AC or battery) into regulated DC voltage suitable for electronic components. A stable power supply ensures accurate sensor readings and reliable system performance. In advanced setups, this module can include voltage regulators, transformers, and even solar panels for renewable energy support. Proper power management enhances the durability and efficiency of the entire irrigation system.

4.TESTING AND IMPLEMENTATION

4.1 TESTING

Testing is an important phase in the development of the IoT system using ESP8266 connected to the Blynk platform. The testing process ensures that both hardware and software components function correctly and communicate reliably over the internet. Initially, individual components such as the ESP8266 module, sensors, and output devices (LED, buzzer, etc.) were tested separately to verify proper connections and functionality. Sensor readings were checked to confirm accurate data acquisition.

Next, Wi-Fi connectivity testing was performed to ensure that the ESP8266 could successfully connect to a stable wireless network. The communication between the ESP8266 and the Blynk cloud server was verified by monitoring real-time data updates on the Blynk mobile application. Notification testing was also conducted to confirm that alerts were received instantly when triggered.

Integration testing was carried out to validate the complete system workflow, ensuring smooth interaction between hardware, cloud platform, and mobile interface. Finally, reliability and performance testing were performed by running the system continuously to check stability, response time, and error handling under different conditions.

The objective of testing is as follows:

- To verify proper communication between the ESP8266 module and the Blynk cloud platform over Wi-Fi.
- To ensure accurate sensor data transmission and reliable real-time alerts through the Blynk mobile application.

Testing Issues

- The ESP8266 may face unstable or weak internet connection, causing delays or failure in sending data to the Blynk server.
- Insufficient or unstable power can lead to unexpected restarts or malfunction of the ESP8266 module.
- Incorrect wiring or improper calibration may result in inaccurate sensor readings or false triggers.
- Network latency or Blynk server downtime may cause delayed notifications or temporary data synchronization issues.

4.1.1 Testing Methodologies

Testing methodologies for the IoT system using ESP8266 connected to the Blynk platform follow a systematic approach to ensure reliability, accuracy, and performance. The first methodology is Unit Testing, where individual components such as the ESP8266 module, sensors, relays, LEDs, and buzzer are tested independently to verify proper functionality. Each hardware component is checked for correct wiring and response.

The second methodology is Connectivity Testing, which ensures that the ESP8266 successfully connects to the Wi-Fi network and communicates with the Blynk cloud server. Data transmission between the device and the mobile application is monitored for consistency and speed.

The third methodology is Integration Testing, where all hardware and software components are combined and tested as a complete system. This verifies smooth interaction between sensors, microcontroller, cloud platform, and user interface.

Finally, Performance and Reliability Testing is conducted by running the system continuously under different network conditions to evaluate stability, response time, and error handling. These methodologies ensure the IoT system operates efficiently and reliably in real-world environments.

4.2 TYPES OF TESTING

System Testing

System testing is performed to evaluate the complete IoT system using ESP8266 connected to the Blynk platform as a fully integrated unit. In this phase, all hardware components such as sensors, actuators, and the ESP8266 module are connected and tested together with the software and cloud environment. The main objective is to ensure that the entire system works according to the project requirements.

During system testing, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk application using the authentication token. Sensor inputs are triggered to verify whether the data is correctly processed and transmitted to the Blynk dashboard in real time. Output devices such as LEDs or buzzers are tested to confirm proper activation based on commands or sensor values.

Notification testing is also conducted to ensure alerts are received without delay. The system is observed under continuous operation to check stability, response time, and error handling. This testing ensures the IoT solution functions reliably in real-time environments.

Unit Testing

Unit testing is performed to verify the proper functioning of each individual component in the IoT system using ESP8266 connected to the Blynk platform. In this phase, every hardware and software unit is tested separately before integrating them into the complete system. The ESP8266 module is first tested to ensure it powers on correctly and establishes a stable Wi-Fi connection. Basic programs such as LED blinking are uploaded to confirm proper board configuration and communication with the Arduino IDE.

Next, individual sensors are tested to check whether they provide accurate readings and respond correctly to environmental changes. Output components such as LEDs, buzzers, or relays are also tested independently to ensure they activate when triggered. Additionally, Blynk authentication tokens and virtual pins are verified to confirm proper data transmission between the ESP8266 and the Blynk mobile application.

By testing each module independently, errors can be identified and resolved early, ensuring smooth integration and reliable system performance in later stages.

Integration Testing

Integration testing is performed to verify the proper interaction between all hardware and software components in the IoT system using ESP8266 connected to the Blynk platform. In this phase, individual modules such as sensors, actuators, ESP8266 microcontroller, Wi-Fi network, and the Blynk mobile application are combined and tested as a complete system.

The testing process begins by connecting sensors and output devices to the ESP8266 and uploading the integrated program. Once powered on, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk cloud server using the authentication token. Sensor data is monitored in real-time on the Blynk application to confirm successful communication between the hardware and cloud platform.

Alerts, notifications, and device control features are tested to ensure proper synchronization. Any delay in data transmission or mismatch in readings is identified and corrected. Integration testing ensures seamless coordination among components and confirms that the system operates reliably in real-world IoT environments.

Validation Testing

Validation testing is performed to ensure that the IoT system using ESP8266 connected to the Blynk platform meets the intended project requirements and functions as expected in real-time conditions. This process verifies whether the complete system satisfies user needs and operational objectives.

Initially, the system is tested by triggering input devices such as sensors or switches to confirm that the ESP8266 correctly processes the input data. The output responses, such as LED indication, buzzer alerts, or relay activation, are observed to validate proper hardware functionality. Simultaneously, the Blynk mobile application is monitored to ensure that real-time data updates and notifications are received without delay.

Further validation includes testing the system under different network conditions to confirm stable Wi-Fi connectivity and reliable cloud communication. Multiple test cases are executed to verify accuracy, response time, and consistency. Continuous operation testing is also conducted to ensure long-term stability. Through validation testing, the system's performance, reliability, and user requirements compliance are successfully confirmed.

System Implementation

System implementation testing is carried out after completing the development and integration of the IoT system using ESP8266 with the Blynk platform. This phase ensures that the fully implemented system operates correctly in a real-time environment. Initially, the hardware components such as sensors, ESP8266 module, and output devices are assembled and verified for proper physical connections. The firmware is then uploaded to the ESP8266 using the Arduino IDE.

Once deployed, the system is connected to a Wi-Fi network and linked to the Blynk cloud using the assigned authentication token. Real-time testing is performed by triggering sensors and observing whether the corresponding data is accurately updated in the Blynk mobile application. Notifications and control commands are tested to confirm two-way communication between the device and the user interface.

The system is also monitored for stability by running it continuously to detect issues such as disconnections, delays, or hardware failures. This testing ensures the implemented IoT solution functions reliably under practical conditions.

5. CONCLUSION AND FUTURE ENHANCEMENT

5.1. Conclusion

The Automatic Irrigation System Using Rain Sensor and Soil Moisture Sensor provides an efficient and reliable solution for modern agriculture. By automating the irrigation process, it reduces water wastage and minimizes manual effort. The integration of soil moisture and rain sensors ensures that crops receive the right amount of water at the right time. This improves crop health, increases productivity, and supports sustainable farming practices. The system is cost-effective, energy-efficient, and easy to implement in small and large agricultural fields. Overall, the project demonstrates how smart technology can enhance agricultural efficiency and contribute to water conservation.

5.2. Future Enhancement

In the future, this system can be enhanced by integrating IoT technology for remote monitoring and control through mobile applications. Farmers can receive real-time updates about soil conditions and irrigation status. Weather forecasting data can be integrated to make predictive irrigation decisions. Solar power can be used to make the system energy independent. Additional sensors such as temperature and humidity sensors can be included for better crop monitoring. The system can also be expanded with data analytics to optimize water usage patterns. These enhancements will transform the irrigation system into a fully smart and intelligent agricultural solution.

BIBLIOGRAPHY

Reference books:

- Banzi, M., & Shiloh, M. (2014). *Getting Started with Arduino*. Maker Media.
- Monk, S. (2016). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.
- Blum, J. (2019). *Exploring Arduino: Tools and Techniques for Engineering Wizardry*. Wiley.
- Kolban, N. (2018). *Kolban's Book on ESP8266*. Leanpub.
- Richardson, M., & Wallace, S. (2012). *Getting Started with Raspberry Pi*. O'Reilly Media.
- McEwen, A., & Cassimally, H. (2014). *Designing the Internet of Things*. Wiley.
- Bahga, A., & Madiseti, V. (2015). *Internet of Things: A Hands-On Approach*. Universities Press.
- Minoli, D. (2017). *Building the Internet of Things with IPv6 and MIPv6*. Wiley.
- Reese, G. (2018). *Cloud Application Architectures*. O'Reilly Media.
- Monk, S. (2018). *Arduino IoT Cloud for Developers*. Packt Publishing.
- Wilmshurst, T. (2010). *Designing Embedded Systems with PIC Microcontrollers*. Newnes.

Website References

- <https://blynk.io>
- <https://www.espressif.com/en/products/socs/esp8266>
- <https://www.arduino.cc>
- <https://randomnerdtutorials.com>
- <https://www.iotforall.com>

APPENDIX

Symbols of Data Flow Diagram



Rectangle Shape

Entity



Rounded Rectangle

Process



Arrow Head

Data Flow



Open Rectangle

Data Store

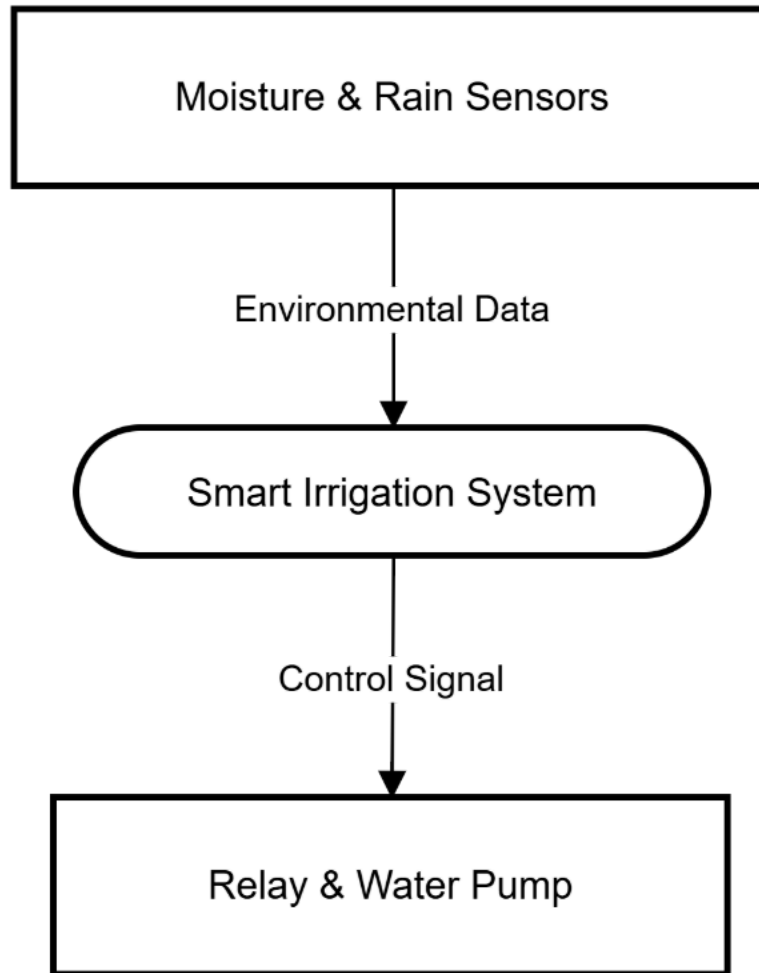


Curved Edge Button

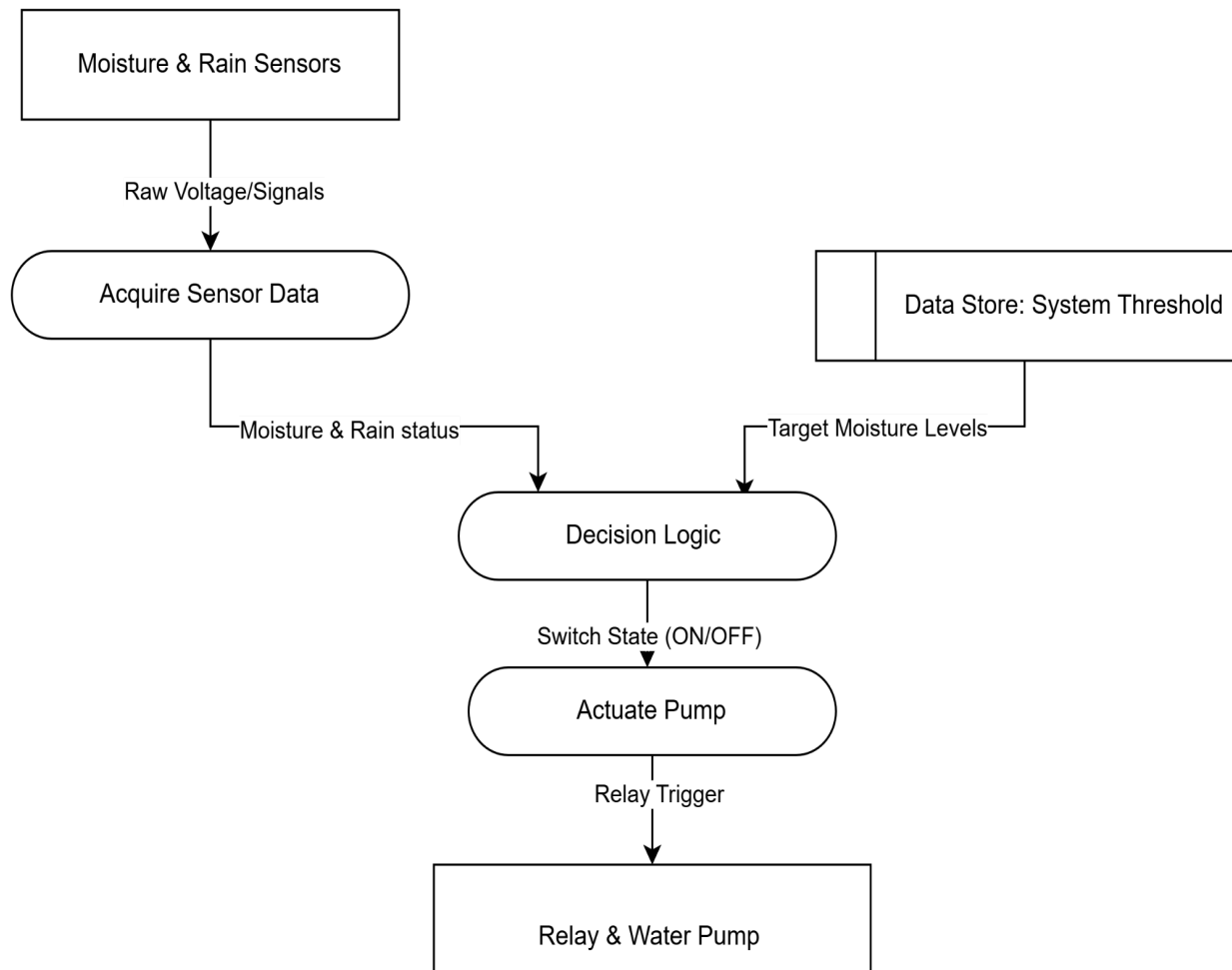
Reports

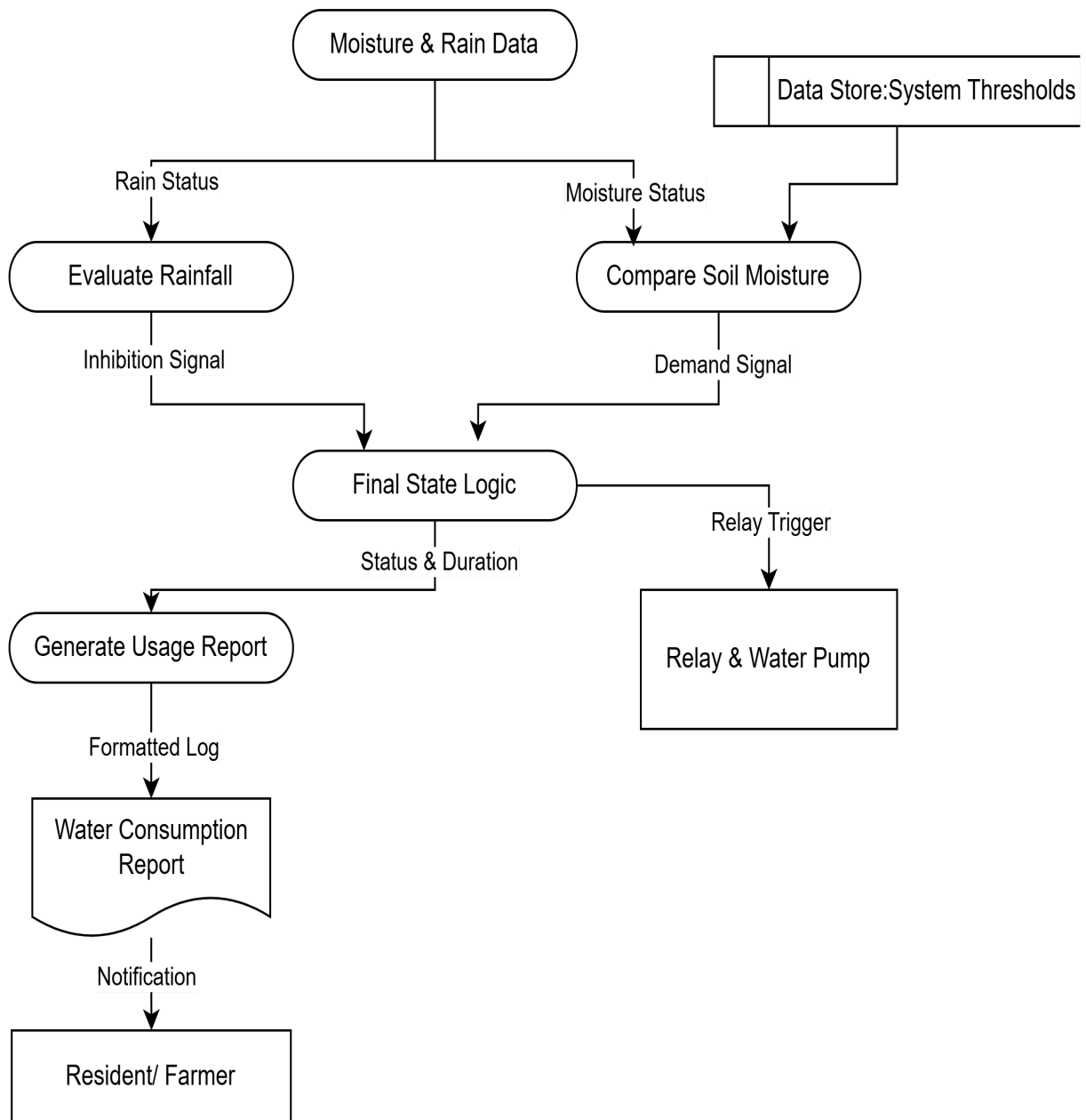
A. Data Flow Diagram

Level: 0



Level: 1



Level: 2

B. Working Diagram:

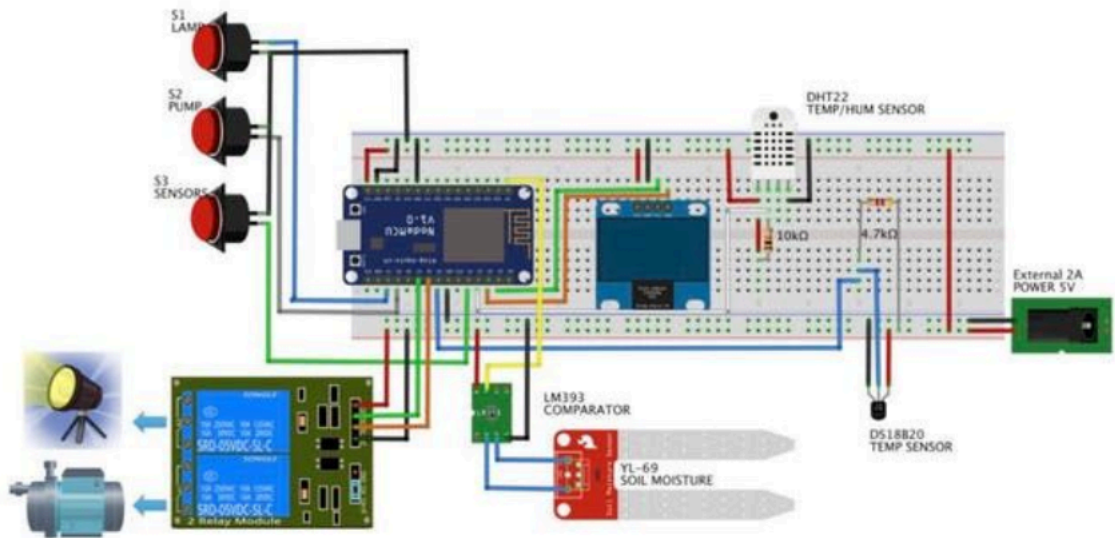


Fig 4: Working diagram of the Automatic Irrigation System showing the interconnection of sensors, Arduino controller, relay module, and water pump for automatic irrigation control.

C. Sample Source code

```
# AUTOMATIC IRRIGATION SYSTEM - ENTERPRISE IoT FIRMWARE
# Device: ESP8266
# Cloud: Blynk IoT
# Version: 4.0.0
# Author: Prasanna Smart AgriTech Systems
```

```
import network
import time
import machine
import urequests
import ujson
import os
import gc
from machine import Pin, ADC, Timer, WDT
```

SYSTEM CONSTANTS :

```
FIRMWARE_VERSION = "4.0.0"
DEVICE_NAME = "SMART_IRRIGATION_NODE_01"
DEBUG_MODE = True

CONFIG_FILE = "irrigation_config.json"
LOG_FILE = "irrigation_logs.txt"
```

```
BLYNK_AUTH = "YOUR_BLYNK_AUTH_TOKEN"
```

```
SSID = "YOUR_WIFI_NAME"
```

```
PASSWORD = "YOUR_WIFI_PASSWORD"
```

```
# Hardware Pins
```

```
SOIL_SENSOR_PIN = 0      # ADC0
```

```
RAIN_SENSOR_PIN = 14     # D5
```

```
PUMP_RELAY_PIN = 12      # D6
```

```
STATUS_LED_PIN = 2       # Built-in LED
```

```
# Thresholds
```

```
SOIL_DRY_THRESHOLD = 600
```

```
SOIL_WET_THRESHOLD = 350
```

```
RAIN_DETECTED_LEVEL = 0
```

```
WATERING_DURATION = 10    # seconds
```

```
STATUS_INTERVAL = 15
```

```
WIFI_CHECK_INTERVAL = 20
```

```
HEARTBEAT_INTERVAL = 30
```

```
MEMORY_CHECK_INTERVAL = 25
```

LOGGER:

```
class Logger:
```

```
    def log(self, level, message):
```

```
        timestamp = time.time()
```

```
entry = "[{}][{}] {}".format(timestamp, level, message)
```

```
if DEBUG_MODE:
```

```
    print(entry)
```

```
try:
```

```
    with open(LOG_FILE, "a") as f:
```

```
        f.write(entry + "\n")
```

```
except:
```

```
    pass
```

```
logger = Logger()
```

CONFIG MANAGER :

```
class ConfigManager:
```

```
    def __init__(self):
```

```
        self.data = {
```

```
            "system_enabled": True,
```

```
            "manual_override": False,
```

```
            "watering_count": 0,
```

```
            "total_water_time": 0
```

```
        }
```

```
    def load(self):
```

```
        try:
```

```
        with open(CONFIG_FILE, "r") as f:
            self.data = ujson.load(f)
            logger.log("INFO", "Config Loaded")
    except:
        self.save()

    def save(self):
        try:
            with open(CONFIG_FILE, "w") as f:
                ujson.dump(self.data, f)
        except:
            logger.log("ERROR", "Config Save Failed")

config = ConfigManager()
config.load()
```

WIFI MANAGER:

```
class WiFiManager:

    def __init__(self):
        self.wlan = network.WLAN(network.STA_IF)
        self.wlan.active(True)

    def connect(self):
        if not self.wlan.isconnected():
            logger.log("INFO", "Connecting WiFi...")
```

```
self.wlan.connect(SSID, PASSWORD)

timeout = 0

while not self.wlan.isconnected() and timeout < 20:

    time.sleep(1)

    timeout += 1


if self.wlan.isconnected():

    logger.log("INFO", "WiFi Connected")

else:

    logger.log("ERROR", "WiFi Failed")


def check(self):

    if not self.wlan.isconnected():

        logger.log("WARN", "WiFi Lost. Reconnecting...")

        self.connect()


def rssi(self):

    try:

        return self.wlan.status("rssi")

    except:

        return 0


wifi = WiFiManager()

wifi.connect()
```

BLYNK CLIENT :

```
class BlynkClient:
```

```
BASE = "http://blynk.cloud/external/api"
```

```
def update(self, pin, value):  
    url = "{} /update?token={} & {}={}".format(self.BASE, BLYNK_AUTH, pin, value)  
    try:  
        r = urequests.get(url)  
        r.close()  
    except:  
        logger.log("ERROR", "Blynk Update Failed")
```

```
def log_event(self, event, message):  
    url = "{} /logEvent?token={} & event={} & description={}".format(  
        self.BASE, BLYNK_AUTH, event, message)  
    try:  
        r = urequests.get(url)  
        r.close()  
    except:  
        logger.log("ERROR", "Blynk Event Failed")
```

```
blynk = BlynkClient()
```

HARDWARE LAYER:

```
class Hardware:
```

```
    def __init__(self):
```

```
self.soil_sensor = ADC(0)
self.rain_sensor = Pin(RAIN_SENSOR_PIN, Pin.IN)
self.pump = Pin(PUMP_RELAY_PIN, Pin.OUT)
self.status_led = Pin(STATUS_LED_PIN, Pin.OUT)
```

```
self.pump.off()
```

```
def read_soil(self):
    return self.soil_sensor.read()
```

```
def is_raining(self):
    return self.rain_sensor.value() == RAIN_DETECTED_LEVEL
```

```
def pump_on(self):
    self.pump.on()
    self.status_led.on()
```

```
def pump_off(self):
    self.pump.off()
    self.status_led.off()
```

```
hardware = Hardware()
```

IRRIGATION ENGINE:

```
class IrrigationEngine:
```

```
def __init__(self):
    self.last_watering = 0

def evaluate(self):
    if not config.data["system_enabled"]:
        return

    if config.data["manual_override"]:
        self.start_watering("Manual Override")
        return

    soil = hardware.read_soil()
    raining = hardware.is_raining()

    logger.log("INFO", "Soil: {} | Rain: {}".format(soil, raining))

    if raining:
        logger.log("INFO", "Rain Detected. Skipping Irrigation")
        hardware.pump_off()
        return

    if soil > SOIL_DRY_THRESHOLD:
        self.start_watering("Soil Dry")

    elif soil < SOIL_WET_THRESHOLD:
        hardware.pump_off()
```

```
def start_watering(self, reason):
    logger.log("WARN", "Watering Started - {}".format(reason))
    blynk.log_event("watering_event", reason)

    hardware.pump_on()
    time.sleep(WATERING_DURATION)
    hardware.pump_off()

    config.data["watering_count"] += 1
    config.data["total_water_time"] += WATERING_DURATION
    config.save()
```

```
engine = IrrigationEngine()
```

HEALTH MONITOR :

```
class HealthMonitor:
```

```
    def memory_check(self):
        free = gc.mem_free()
        logger.log("DEBUG", "Free Memory: {}".format(free))
        if free < 5000:
            logger.log("ERROR", "Low Memory - Resetting")
            machine.reset()

    def diagnostics(self):
        logger.log("INFO", "Firmware: {}".format(FIRMWARE_VERSION))
```

```
logger.log("INFO", "Device: {}".format(DEVICE_NAME))
logger.log("INFO", "RSSI: {}".format(wifi.rssi()))
logger.log("INFO", "Water Cycles: {}".format(config.data["watering_count"]))
```

```
health = HealthMonitor()
```

```
wdt = WDT(timeout=8000)
```

SCHEDULER:

```
class Scheduler:
```

```
    def __init__(self):
        self.last_status = 0
        self.last_wifi = 0
        self.last_memory = 0
        self.last_heartbeat = 0
```

```
    def run(self):
        now = time.time()

        if now - self.last_status > STATUS_INTERVAL:
            self.last_status = now
            self.send_status()
```

```
if now - self.last_wifi > WIFI_CHECK_INTERVAL:
```

```
    self.last_wifi = now
```

```
    wifi.check()
```

```
if now - self.last_memory > MEMORY_CHECK_INTERVAL:
```

```
    self.last_memory = now
```

```
    health.memory_check()
```

```
if now - self.last_heartbeat > HEARTBEAT_INTERVAL:
```

```
    self.last_heartbeat = now
```

```
    blynk.update("V6", "ONLINE")
```

```
def send_status(self):
```

```
    blynk.update("V1", hardware.read_soil())
```

```
    blynk.update("V2", config.data["watering_count"])
```

```
    blynk.update("V3", wifi.rssi())
```

```
    logger.log("INFO", "Status Updated")
```

```
scheduler = Scheduler()
```

MAIN LOOP:

```
logger.log("INFO", "Smart Irrigation System Booted")
```

```
health.diagnostics()
```

```
while True:
```

```
    try:
```

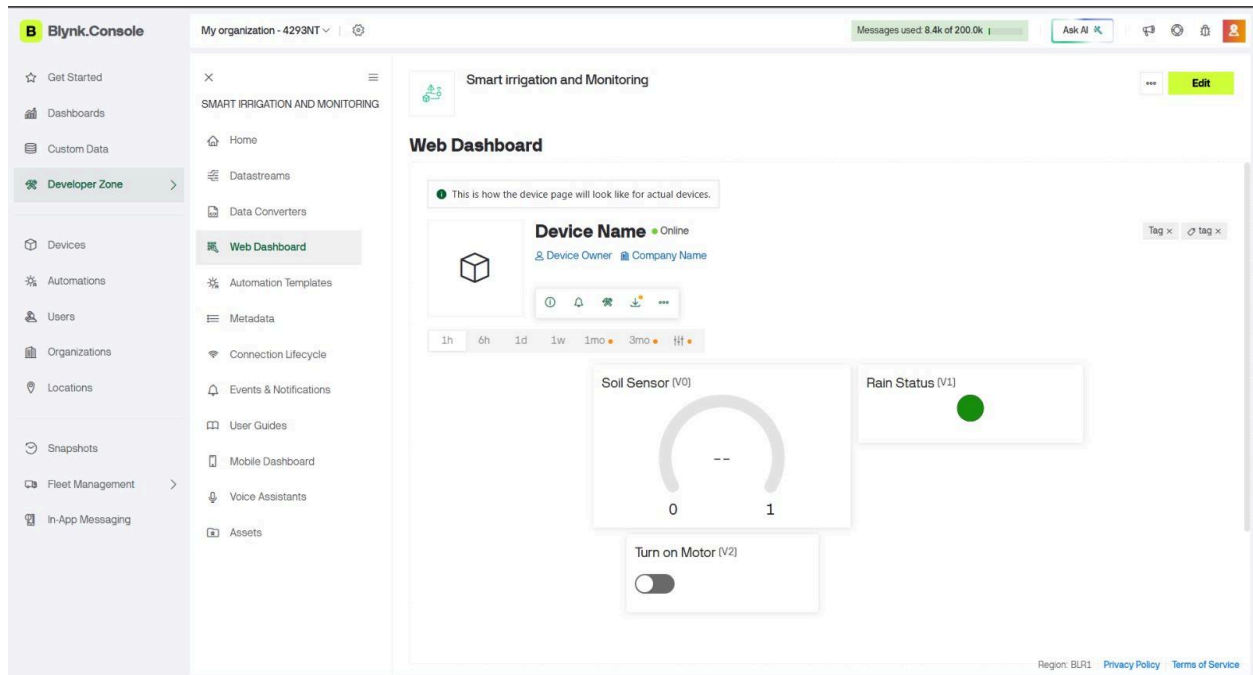
```
engine.evaluate()  
scheduler.run()  
wdt.feed()  
time.sleep(5)
```

```
except Exception as e:
```

```
    logger.log("ERROR", "Runtime Error {}".format(e))  
    time.sleep(2)
```

D. Sample Screens:

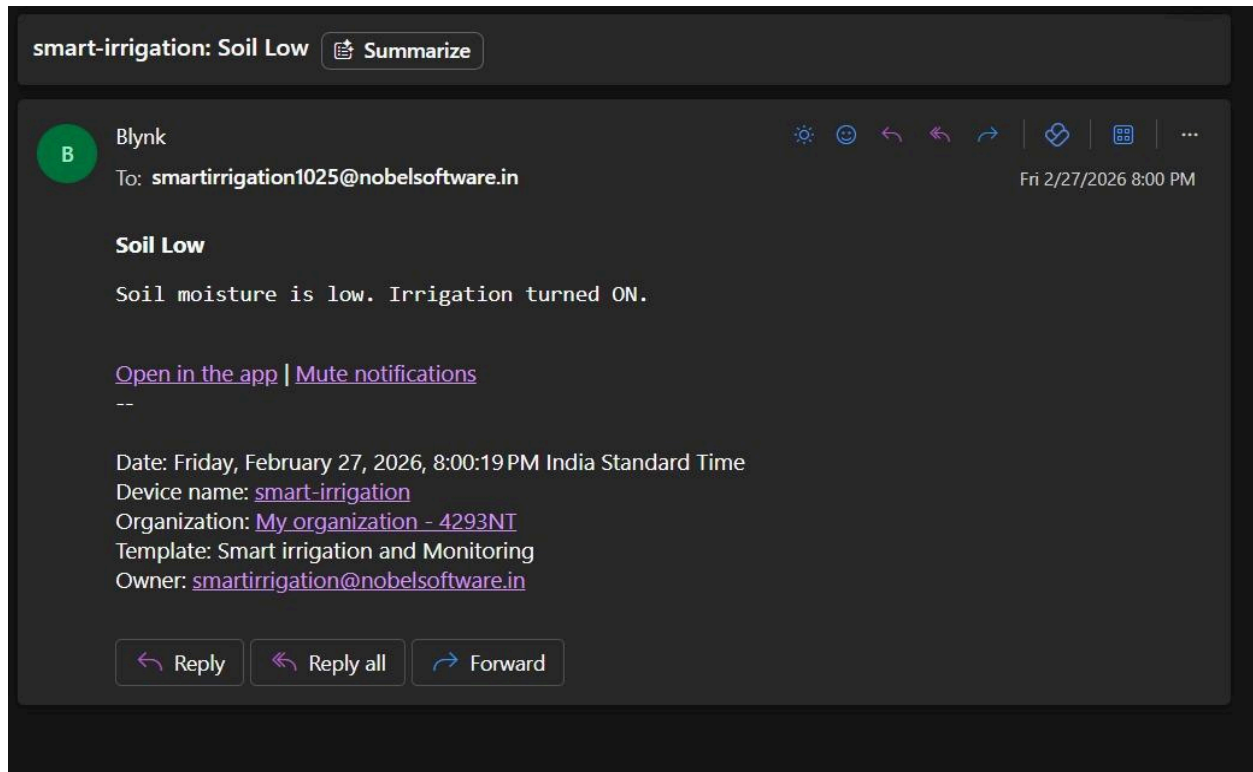
Web Dashboard



Form Description:

This screen shows the dashboard of blynk app which shows soil moisture level and rain status.

Received Mail



Form Description:

This screen shows the mail received from the blynk app to the user.